Session 11B Homework — Delete, Verified Against the Real Database

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, both Projects and Workspaces are fully MongoDB-backed, with real filtering, and you've independently confirmed that through `mongosh` and MongoDB Compass. One CRUD operation is still missing on both: there is no way to remove a project or workspace. This homework closes that gap, reusing only what class already built — `ParseIntPipe`, the not-found pattern every `findOne` / `findByKey` already uses, and one new, self-explanatory Mongoose method — and asks you to prove each delete directly against the real database, not just through the API's own response.

Goal

A fully CRUD-complete backend: both resources support create, read, update, AND delete, all persisted in MongoDB, with every delete verified by a real before/after database check.

What to build

A complete `DELETE /projects/:id` route, verified with a real before/after `mongosh` check (CORE); the identical route on `DELETE /workspaces/:id`, with the same real check (STRETCH); and three written reasoning answers — one as a short code comment above `ProjectsService`'s own `remove()`, two as short written notes — on why this app queries by `id`, why a direct MongoDB write bypasses Nest's own validation, and why a restart no longer resets data (CHALLENGE).

CORE (required)

Add `DELETE /projects/:id` to `ProjectsController` / `ProjectsService`:

- `@Delete(':id')` with `@Param('id', ParseIntPipe) id: number` — the identical pipe already used on `findOne` / `update`.
- In the service, `remove(id)` uses `this.model.findOneAndDelete({ id })` — the same shape as `findOne`, except it also removes the match. If nothing matched, throw the same `NotFoundException` pattern already used everywhere else in this file.
- Verify it with a real BEFORE/AFTER check in `mongosh`: query for the document before deleting it (it's there), delete it through the API, then run the identical `mongosh` query again (it's gone).

STRETCH (required)

Add the identical `DELETE /workspaces/:id` to `WorkspacesController` / `WorkspacesService` — same pattern, independently applied, with the same real before/after `mongosh` check on the `workspaces` collection.

CHALLENGE (required)

Write three short answers (comments or a short written note, as instructed by each question):

1. Directly above `ProjectsService`'s own `remove()`, a short code comment explaining why every method in this service queries by the app's own `id` field, never MongoDB's own `_id` — what would go wrong if even one method queried by `_id` instead?
2. In a short written answer (not code): if you inserted a document directly into MongoDB through `mongosh` or Compass — skipping the API entirely — would Nest's own `ValidationPipe` or any DTO decorator ever see it? Why or why not?
3. In a short written answer (not code): in your own words, why does restarting the Nest server no longer reset the data, when it always did before this session?

Rules

Allowed: everything class already used — `ParseIntPipe`, `findOne` / `findOneAndDelete`, the `NotFoundException` pattern, async/await, `mongosh` queries. One new Mongoose method (`findOneAndDelete`) is the only new syntax. **Not yet:** no soft-delete, no cascading delete of related data, no confirmation step, no guards.

Test it

BASH

```
curl -i -X DELETE localhost:3000/projects/999
mongosh --eval 'db.getSiblingDB("b4f-board").projects.find({ key: "TEMP" })'
curl -X POST localhost:3000/projects -H "Content-Type: application/json" -d
'{"name":"Temp","key":"TEMP"}'
# read the "id" field from the JSON response above — it will NOT reliably be 4; it depends
on how
# many projects already exist for real in your own database. Use YOUR OWN real id below.
mongosh --eval 'db.getSiblingDB("b4f-board").projects.find({ key: "TEMP" })'
curl -i -X DELETE localhost:3000/projects/<the id you just recorded>
mongosh --eval 'db.getSiblingDB("b4f-board").projects.find({ key: "TEMP" })'
curl -i -X GET localhost:3000/projects/<the same id>
curl -i -X DELETE localhost:3000/projects/abc
```

Done when

- `DELETE /projects/999` (doesn't exist) returns a clean `404`.

- Creating a project, then deleting it by its real id, returns `200` with the deleted project's own data.
- The before-delete `mongosh` query shows the real document present; the identical after-delete query shows it genuinely gone from MongoDB itself — not just absent from a follow-up `GET`.
- A `GET` on that same id afterward returns `404` — it is really gone, and still gone after a server restart.
- `DELETE /projects/abc` (malformed id) returns a clean `400` — `ParseIntPipe` protects this route too.
- The identical behavior, with the identical real before/after `mongosh` check, holds for `DELETE /workspaces/:id`.
- All three CHALLENGE answers explain WHY, not just WHAT — the rationale comment explains why querying by `id` consistently matters, the second answer explains why a direct MongoDB write bypasses the Nest validation layer entirely, and the third answer explains the real reason restart no longer resets data (two separate running processes, only one of which restarts).

Working time

About 90–120 minutes for a normal student — CORE and STRETCH are the same pattern applied twice, each with a real database check; CHALLENGE is three short written answers, not new code.